

Student Course Portal

Angular (v20.0) Complete Hands-On Solutions Manual

Digital Nurture 5.0 Deep Skilling Program

Track: .NET Full Stack Engineer Track

Framework Version: Angular v20.0 (Standalone Pattern Architecture)

Contents: Exercises 1 to 10 End-to-End Complete Implementation

Date: July 2026

HANDS-ON 1: Environment Setup, Project Structure & First Component

Task 1: Scaffold and Explore the Angular Project

To initialize the unified project workspace for the Student Course Portal under Angular v20, execute the following CLI commands:

```
ng new student-course-portal --routing --style=css
cd student-course-portal
ng serve
```

File Explanations (notes.txt)

Below is the structured breakdown of the primary configuration and bootstrap files generated within the modern standalone workspace architecture:

File Path	Purpose and Technical Role in Angular v20 Workspace
angular.json	CLI workspace configuration file containing build, test, and execution profiles for all projects in the workspace.
tsconfig.json	Root TypeScript compiler configuration detailing broad language feature flags, target profiles, and path mappings.
tsconfig.app.json	Application-specific TypeScript configurations overriding defaults to compile browser binaries, specifying file inclusions.
package.json	Node project manifest specifying third-party package dependencies (e.g., Angular core packages), metadata, and script targets.
src/main.ts	The absolute programmatic entry point that initializes the runtime execution environment and bootstraps the root application components.
src/app/app.config.ts	Modern standalone application provider configurations containing environment configurations, route listings, and global dependencies.
src/app/app.component.ts	The main root component shell that hosts all other UI component child hierarchies and serves as the primary template framework.
src/index.html	The main single-page HTML delivery container hosting the execution canvas where Angular mounts the compiled root DOM component tree.

Budget Analysis under angular.json

Within `angular.json` under `architect > build > configurations > production`, bundle performance criteria are controlled via specific budget objects:

```
"budgets": [  
  {  
    "type": "initial",  
    "maximumWarning": "500kb",  
    "maximumError": "1mb"  
  }  
]
```

Technical Function: Budget properties set strict constraints on bundle build outputs. If code changes cause production distribution bundles to grow beyond `maximumWarning`, compilation warns the developer. If they exceed `maximumError`, the production build explicitly fails, protecting clients from loading overly large JavaScript binaries over mobile networks.

Task 2: Create and Organize Components

Generate the layout shell and core view structures using the Angular CLI:

```
ng generate component components/header  
ng generate component pages/home  
ng generate component pages/course-list  
ng generate component pages/student-profile
```

src/app/components/header/header.component.html

```
<nav class="navbar">  
  <span class="brand-title">Student Course Portal</span>  
  <div class="nav-links">  
    <a href="#">Home</a>  
    <a href="#">Courses</a>  
    <a href="#">Profile</a>  
  </div>  
</nav>
```

src/app/pages/home/home.component.html

```
<div class="home-container">  
  <h1>Welcome to the Student Course Portal</h1>  
  <p>Manage your academic records, enroll in dynamic classes, monitor credit weights, and track final grade markers instantly.</p>  
  
  <div class="stats-row">  
    <div class="stat-card">  
      <h4>Courses Available</h4>  
      <p class="stat-value">12</p>  
    </div>  
    <div class="stat-card">  
      <h4>Enrolled</h4>  
      <p class="stat-value">3</p>  
    </div>  
    <div class="stat-card">  
      <h4>GPA</h4>  
      <p class="stat-value">3.8</p>  
    </div>  
  </div>  
</div>
```

```
    </div>
  </div>
</div>
```

src/app/app.component.html

```
<app-header></app-header>
<main class="app-viewport">
  <app-home></app-home>
  <router-outlet></router-outlet>
</main>
```

Architecture Note: Angular v20 Standalone Registration

In Angular v20 standalone projects, CLI generated components specify `standalone: true` natively. They must be explicitly imported inside the `imports: [...]` metadata array of any sibling standalone component that uses their selector.

HANDS-ON 2: Data Binding, Lifecycle Hooks & Component Communication

Task 1: All Four Binding Types

Demonstrates data management using string interpolation, unidirectional property binding, event bindings, and bidirectional forms integration via `ngModel`.

src/app/pages/home/home.component.ts

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-home',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './home.component.html',
  styleUrls: ['./home.component.css']
})
export class HomeComponent {
  portalName: string = 'Student Course Portal';
  isPortalActive: boolean = true;
  message: string = '';
  searchTerm: string = '';

  onEnrollClick(): void {
    this.message = 'Enrollment opened!';
  }
}
```

src/app/pages/home/home.component.html (Updated)

```
<div class="home-container">
  <!-- 1. String Interpolation -->
  <h1>{{ portalName }}</h1>

  <!-- 2. Property Binding -->
  <button [disabled]="!isPortalActive" (click)="onEnrollClick()">Enroll Now</button>

  <!-- 3. Event Binding Feedback -->
  <p *ngIf="message" class="alert-info">{{ message }}</p>

  <!-- 4. Two-Way Data Binding -->
  <div class="search-box">
    <label for="search">Search Courses:</label>
    <input id="search" type="text" [(ngModel)]="searchTerm" placeholder="Enter topic..." />
    <p>Searching for: <span class="highlight">{{ searchTerm }}</span></p>
  </div>
</div>
```

Core Concept: Unidirectional [Property] vs Bidirectional [(ngModel)]

[Property]: Direct data flow from the TypeScript state engine into the target DOM element property. Component state changes mutate the view, but user interactions on the UI cannot natively mutate the backing state.

[(ngModel)]: Syncs data both ways. It is a syntactic shorthand combining property binding `[(ngModel)]="prop"` and event notification `(ngModelChange)="prop=$event"`, instantly synchronizing user interface inputs with the backing data store.

Task 2: Lifecycle Hooks

src/app/components/course-card/course-card.component.ts

```
import { Component, Input, OnInit, OnDestroy, OnChanges, SimpleChanges, Output, EventEmitter }
from '@angular/core';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-course-card',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './course-card.component.html',
  styleUrls: ['./course-card.component.css']
})
export class CourseCardComponent implements OnInit, OnDestroy, OnChanges {
  @Input() course: any;
  @Output() enrollRequested = new EventEmitter<number>();

  ngOnChanges(changes: SimpleChanges): void {
    console.log('CourseCardComponent input modified:', changes);
    if (changes['course']) {
      const prev = changes['course'].previousValue;
      const curr = changes['course'].currentValue;
      console.log(`Course changed from ${prev ? prev.name : 'None'} to ${curr.name}`);
    }
  }

  ngOnInit(): void {
    console.log('CourseCardComponent loaded instance.');
```

Task 3: @Input and @Output - Parent-Child Communication

src/app/components/course-card/course-card.component.html

```
<div class="card-frame">
  <h3>{{ course.name }}</h3>
```

```

<p><strong>Code:</strong> {{ course.code }}</p>
<p><strong>Credits:</strong> {{ course.credits }}</p>
<button class="btn-enroll" (click)="enrollRequested.emit(course.id)">Enroll</button>
</div>

```

src/app/pages/course-list/course-list.component.ts

```

import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { CourseCardComponent } from '../../components/course-card/course-card.component';

@Component({
  selector: 'app-course-list',
  standalone: true,
  imports: [CommonModule, CourseCardComponent],
  templateUrl: './course-list.component.html',
  styleUrls: ['./course-list.component.css']
})
export class CourseListComponent implements OnInit {
  courses: any[] = [];
  selectedCourseId: number | null = null;

  ngOnInit(): void {
    this.courses = [
      { id: 101, name: 'Advanced Angular Architecture', code: 'CS-A20', credits: 4,
        gradeStatus: 'passed' },
      { id: 102, name: 'Enterprise .NET Core WebAPIs', code: 'CS-N50', credits: 3, gradeStatus:
        'pending' },
      { id: 103, name: 'Relational Database Engineering', code: 'DB-M22', credits: 4,
        gradeStatus: 'failed' },
      { id: 104, name: 'Cloud Native Microservices', code: 'CL-A09', credits: 3, gradeStatus:
        'pending' },
      { id: 105, name: 'Data Structures and Algorithms', code: 'DS-G10', credits: 4,
        gradeStatus: 'passed' }
    ];
  }

  onEnroll(courseId: number): void {
    console.log('Enrolling in course: ' + courseId);
    this.selectedCourseId = courseId;
  }
}

```

src/app/pages/course-list/course-list.component.html

```

<div class="catalog-view">
  <h2>Available Curriculum Catalog</h2>

  <div class="card-grid">
    <app-course-card
      *ngFor="let c of courses"
      [course]="c"
      (enrollRequested)="onEnroll($event)">
    </app-course-card>
  </div>

  <div *ngIf="selectedCourseId" class="selection-footer">

```

```
<p>Selected course ID: <strong>{{ selectedCourseId }}</strong></p>  
</div>  
</div>
```


HANDS-ON 3: Directives & Pipes - Built-in and Custom

Task 1: Structural Directives

Implements structural workflows utilizing delayed lifecycle indicators, high-efficiency grid trackers, and structural fallbacks when vectors evaluate to zero length.

src/app/pages/course-list/course-list.component.ts (Updated Lifecycle)

```
isLoading: boolean = true;

ngOnInit(): void {
  setTimeout(() => {
    this.courses = [
      { id: 101, name: 'Advanced Angular Architecture', code: 'CS-A20', credits: 4,
        gradeStatus: 'passed', isEnrolled: false },
      { id: 102, name: 'Enterprise .NET Core WebAPIs', code: 'CS-N50', credits: 3, gradeStatus:
        'pending', isEnrolled: true },
      { id: 103, name: 'Relational Database Engineering', code: 'DB-M22', credits: 4,
        gradeStatus: 'failed', isEnrolled: false }
    ];
    this.isLoading = false;
  }, 1500);
}

trackByCourseId(index: number, course: any): number {
  return course.id;
}
```

src/app/pages/course-list/course-list.component.html (Updated Structural Layout)

```
<div class="catalog-view">
  <div *ngIf="isLoading; else listContainer" class="loading-panel">
    <p>Loading courses...</p>
  </div>

  <ng-template #listContainer>
    <div *ngIf="courses.length > 0; else noCourses" class="card-grid">
      <app-course-card
        *ngFor="let c of courses; let i = index; trackBy: trackByCourseId"
        [course]="c"
        (enrollRequested)="onEnroll($event)">
      </app-course-card>
    </div>
  </ng-template>

  <ng-template #noCourses>
    <div class="empty-state">
      <p>No courses available.</p>
    </div>
  </ng-template>
</div>
```

Performance Optimization: Why trackBy is Essential

By default, `*ngFor` performs identity checks using reference matching. If the backing array is reloaded or updated, Angular tears down and completely rebuilds the entire DOM node hierarchy. Implementing `trackBy` targets specific elements using a unique key identifier (like `course.id`). As a result, only records with modified properties are updated in place, preventing unnecessary layout recalculations.

Task 2: Attribute Directives ngClass and ngStyle

src/app/components/course-card/course-card.component.ts (Updated Class/Style Logic)

```
isExpanded: boolean = false;

get cardClasses() {
  return {
    'card--enrolled': this.course.isEnrolled,
    'card--full': this.course.credits >= 4,
    'expanded': this.isExpanded
  };
}

toggleExpand(): void {
  this.isExpanded = !this.isExpanded;
}
```

src/app/components/course-card/course-card.component.html (Updated Bindings)

```
<div [ngClass]="cardClasses" [ngStyle]="{'border-left': course.gradeStatus === 'passed' ? '6px solid green' : course.gradeStatus === 'failed' ? '6px solid red' : '6px solid grey'}"
class="card-frame">
  <div class="card-header">
    <h3>{{ course.name }}</h3>

    <span [ngSwitch]="course.gradeStatus" class="status-badge">
      <span *ngSwitchCase="'passed'" class="badge-green">Passed</span>
      <span *ngSwitchCase="'failed'" class="badge-red">Failed</span>
      <span *ngSwitchCase="'pending'" class="badge-grey">Pending</span>
    </span>
  </div>

  <p><strong>Code:</strong> {{ course.code }}</p>
  <p><strong>Credits:</strong> {{ course.credits | creditLabel }}</p>

  <div *ngIf="isExpanded" class="extended-details">
    <p>Course Syllabus includes master level deep-dive instruction modules and real world integration laboratories.</p>
  </div>

  <div class="action-buttons">
    <button (click)="toggleExpand()">{{ isExpanded ? 'Hide Details' : 'Show Details' }}</button>
    <button class="btn-enroll" (click)="enrollRequested.emit(course.id)">Enroll</button>
  </div>
</div>
```

```
</div>
</div>
```

Task 3: Custom Directive and Custom Pipe

src/app/directives/highlight.directive.ts

```
import { Directive, ElementRef, HostListener, Input, Renderer2, OnInit } from '@angular/core';

@Directive({
  selector: '[appHighlight]',
  standalone: true
})
export class HighlightDirective implements OnInit {
  @Input() appHighlight: string = 'yellow';

  constructor(private el: ElementRef, private renderer: Renderer2) {}

  ngOnInit(): void {
    // Initialization defaults if required
  }

  @HostListener('mouseenter') onMouseEnter() {
    this.changeBackground(this.appHighlight);
  }

  @HostListener('mouseleave') onMouseLeave() {
    this.changeBackground(null);
  }

  private changeBackground(color: string | null) {
    this.renderer.setStyle(this.el.nativeElement, 'background-color', color);
  }
}
```

src/app/pipes/credit-label.pipe.ts

```
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'creditLabel',
  standalone: true,
  pure: true
})
export class CreditLabelPipe implements PipeTransform {
  transform(value: number | null | undefined): string {
    if (value === null || value === undefined || value === 0) {
      return 'No Credits';
    }
    if (value === 1) {
      return '1 Credit';
    }
    return `${value} Credits`;
  }
}
```

```
}  
}
```

HANDS-ON 4: Template-Driven Forms & Validation

Task 1 & Task 2: Build Form & Apply Validation Rules

Implements user interactions via template directives. Component layouts listen to input mutation tokens and output validation states using default browser styling configurations.

src/app/pages/enrollment-form/enrollment-form.component.ts

```
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { FormsModule, NgForm } from '@angular/forms';

@Component({
  selector: 'app-enrollment-form',
  standalone: true,
  imports: [CommonModule, FormsModule],
  templateUrl: './enrollment-form.component.html',
  styleUrls: ['./enrollment-form.component.css']
})
export class EnrollmentFormComponent {
  submitted: boolean = false;
  formData = {
    studentName: '',
    studentEmail: '',
    courseId: null,
    preferredSemester: 'Odd',
    agreeToTerms: false
  };

  onSubmit(form: NgForm): void {
    console.log('Form evaluation status valid:', form.valid);
    console.log('Captured runtime value:', form.value);
    if (form.valid) {
      this.submitted = true;
    }
  }
}
```

src/app/pages/enrollment-form/enrollment-form.component.html

```
<div class="form-wrapper">
  <h2>Academic Enrollment Request Form</h2>

  <div *ngIf="submitted" class="success-banner">
    Enrollment request submitted successfully!
  </div>

  <form #enrollForm="ngForm" (ngSubmit)="onSubmit(enrollForm)" novalidate>
    <div class="control-group">
      <label>Student Full Name:</label>
      <input type="text" name="studentName" [(ngModel)]="formData.studentName"
        #nameCtrl="ngModel" required minlength="3" class="form-input" />
      <div *ngIf="nameCtrl.touched && nameCtrl.errors" class="error-msg">
```

```

        <span *ngIf="nameCtrl.errors['required']">Name is required.</span>
        <span *ngIf="nameCtrl.errors['minlength']">Name must be at least 3 characters.</span>
    </div>
</div>

<div class="control-group">
    <label>Academic Email Address:</label>
    <input type="email" name="studentEmail" [(ngModel)]="formData.studentEmail"
#emailCtrl="ngModel" required email class="form-input" />
    <div *ngIf="emailCtrl.touched && emailCtrl.errors" class="error-msg">
        <span *ngIf="emailCtrl.errors['required']">Email is required.</span>
        <span *ngIf="emailCtrl.errors['email']">Provide a valid academic email address.</span>
    </div>
</div>

<div class="control-group">
    <label>Target Course Catalog ID:</label>
    <input type="number" name="courseId" [(ngModel)]="formData.courseId" #idCtrl="ngModel"
required class="form-input" />
    <div *ngIf="idCtrl.touched && idCtrl.errors" class="error-msg">
        Course ID target index link is mandatory.
    </div>
</div>

<div class="control-group">
    <label>Preferred Semester Target:</label>
    <select name="preferredSemester" [(ngModel)]="formData.preferredSemester" class="form-
input">
        <option value="Odd">Odd Semester</option>
        <option value="Even">Even Semester</option>
    </select>
</div>

<div class="control-group checkbox-row">
    <input type="checkbox" id="terms" name="agreeToTerms" [(ngModel)]="formData.agreeToTerms"
#termsCtrl="ngModel" requiredTrue />
    <label for="terms">I certify all uploaded registration parameters are valid.</label>
</div>
<div *ngIf="termsCtrl.touched && termsCtrl.errors" class="error-msg">
    You must accept validation certification parameters before processing.
</div>

<div class="form-actions">
    <button type="button" class="btn-reset" (click)="enrollForm.resetForm()">Reset Form</
button>
    <button type="submit" class="btn-submit" [disabled]="enrollForm.invalid">Submit
Application</button>
</div>
</form>
</div>

```

src/app/pages/enrollment-form/enrollment-form.component.css

```

.form-input.ng-invalid.ng-touched {
    border: 1px solid #dc2626;
    background-color: #fef2f2;
}
.form-input.ng-valid.ng-touched {
    border: 1px solid #16a34a;
}

```

```
    background-color: #f0fdf4;
  }
  .error-msg {
    color: #dc2626;
    font-size: 8.5pt;
    margin-top: 2px;
  }
}
```

HANDS-ON 5: Reactive Forms - FormBuilder, FormGroup, FormArray & Custom Validators

Task 1 & Task 2: Reactive Model Design, Custom Sync/Async Validators & FormArray

Defines form structure in the component class, decoupling data structure definitions from presentation templates for enhanced unit testing capabilities.

src/app/pages/reactive-enrollment-form/reactive-enrollment-form.component.ts

```
import { Component, OnInit } from '@angular/core';
import { CommonModule } from '@angular/common';
import { ReactiveFormsModule, FormBuilder, FormGroup, FormArray, FormControl, Validators,
  AbstractControl, ValidationErrors } from '@angular/forms';

@Component({
  selector: 'app-reactive-enrollment-form',
  standalone: true,
  imports: [CommonModule, ReactiveFormsModule],
  templateUrl: './reactive-enrollment-form.component.html',
  styleUrls: ['./reactive-enrollment-form.component.css']
})
export class ReactiveEnrollmentFormComponent implements OnInit {
  enrollForm!: FormGroup;

  constructor(private fb: FormBuilder) {}

  ngOnInit(): void {
    this.enrollForm = this.fb.group({
      studentName: ['', [Validators.required, Validators.minLength(3)]],
      studentEmail: ['', [Validators.required, Validators.email], [this.simulateEmailCheck]],
      courseId: [null, [Validators.required, this.noCourseCode]],
      preferredSemester: ['Odd', Validators.required],
      agreeToTerms: [false, Validators.requiredTrue],
      additionalCourses: this.fb.array([])
    });
  }

  // Typed getter for dynamic FormArray access
  get additionalCourses(): FormArray {
    return this.enrollForm.get('additionalCourses') as FormArray;
  }

  addCourse(): void {
    this.additionalCourses.push(this.fb.control('', Validators.required));
  }

  removeCourse(index: number): void {
    this.additionalCourses.removeAt(index);
  }

  // Custom Synchronous Validator
  noCourseCode(control: AbstractControl): ValidationErrors | null {
```



```

    const val = String(control.value || '');
    if (val.toUpperCase().startsWith('XX')) {
      return { 'noCourseCode': true };
    }
    return null;
  }

  // Custom Asynchronous Validator simulating API latency
  simulateEmailCheck(control: AbstractControl): Promise<ValidationErrors | null> {
    return new Promise((resolve) => {
      setTimeout(() => {
        const email = String(control.value || '');
        if (email.includes('test@')) {
          resolve({ 'emailTaken': true });
        } else {
          resolve(null);
        }
      }, 800);
    });
  }

  onSubmit(): void {
    console.log('Filtered properties subset value (omitting disabled):',
this.enrollForm.value);
    console.log('Absolute runtime comprehensive dataset raw value:',
this.enrollForm.getRawValue());
  }
}

```

src/app/pages/reactive-enrollment-form/reactive-enrollment-form.component.html

```

<div class="form-wrapper">
  <h2>Reactive Enrollment Control Panel</h2>

  <form [formGroup]="enrollForm" (ngSubmit)="onSubmit()">
    <div class="control-group">
      <label>Student Name:</label>
      <input formControlName="studentName" class="form-input" />
      <div *ngIf="enrollForm.get('studentName')?.touched &&
enrollForm.get('studentName')?.invalid" class="error-msg">
        Name configuration criteria mismatch. Minimum length 3 required.
      </div>
    </div>

    <div class="control-group">
      <label>Email Address:</label>
      <input formControlName="studentEmail" class="form-input" />
      <div *ngIf="enrollForm.get('studentEmail')?.touched &&
enrollForm.get('studentEmail')?.errors" class="error-msg">
        <span *ngIf="enrollForm.get('studentEmail')?.errors?.['emailTaken']">This academic
address is already registered.</span>
        <span *ngIf="enrollForm.get('studentEmail')?.errors?.['email']">Invalid syntax schema
formatting.</span>
      </div>
    </div>

    <div class="control-group">
      <label>Target Course ID:</label>
      <input formControlName="courseId" class="form-input" />
    </div>
  </form>
</div>

```

```

    <div *ngIf="enrollForm.get('courseId')?.touched && enrollForm.get('courseId')?.errors"
class="error-msg">
      <span *ngIf="enrollForm.get('courseId')?.errors?.['noCourseCode']">Course code
sequences starting with 'XX' are restricted.</span>
      <span *ngIf="enrollForm.get('courseId')?.errors?.['required']">Course reference
parameter mandatory.</span>
    </div>
  </div>

  <!-- Dynamic Core Segment -->
  <div class="array-section">
    <h4>Simultaneous Elective Registrations</h4>
    <button type="button" (click)="addCourse()" class="btn-add">Add Another Course</button>

    <div formArrayName="additionalCourses" class="array-list">
      <div *ngFor="let ctrl of additionalCourses.controls; let i=index" class="array-item-
row">
        <input [formControlName]="i" placeholder="Elective Catalog Code" class="form-input-
inline" />
        <button type="button" (click)="removeCourse(i)" class="btn-remove">Remove</button>
      </div>
    </div>
  </div>

  <button type="submit" [disabled]="enrollForm.invalid" class="btn-submit">Submit Application
Suite</button>
</form>
</div>

```

Core Architecture: value vs getRawValue()

`FormGroup.value` returns an object aggregating values from active form controls, automatically excluding any disabled components. Conversely, `FormGroup.getRawValue()` skips validation status filters and returns all parameters, ensuring disabled key-value structures are fully retained in backend payloads.

HANDS-ON 6: Services & Dependency Injection

Task 1: Create and Use a Course Service

Centralizes business operations and entities into decoupled data services using the root injector singleton pattern.

src/app/models/course.model.ts

```
export interface Course {
  id: number;
  name: string;
  code: string;
  credits: number;
  gradeStatus: 'passed' | 'failed' | 'pending';
}
```

src/app/services/course.service.ts

```
import { Injectable } from '@angular/core';
import { Course } from '../models/course.model';

@Injectable({
  providedIn: 'root'
})
export class CourseService {
  private courses: Course[] = [
    { id: 1, name: 'Advanced Angular Architecture', code: 'CS-A20', credits: 4, gradeStatus: 'passed' },
    { id: 2, name: 'Enterprise .NET Core WebAPIs', code: 'CS-N50', credits: 3, gradeStatus: 'pending' },
    { id: 3, name: 'Relational Database Engineering', code: 'DB-M22', credits: 4, gradeStatus: 'failed' },
    { id: 4, name: 'Cloud Native Microservices', code: 'CL-A09', credits: 3, gradeStatus: 'pending' },
    { id: 5, name: 'Data Structures and Algorithms', code: 'DS-G10', credits: 4, gradeStatus: 'passed' }
  ];

  getCourses(): Course[] {
    return this.courses;
  }

  getCourseById(id: number): Course | undefined {
    return this.courses.find(c => c.id === id);
  }

  addCourse(course: Course): void {
    this.courses.push(course);
  }
}
```

Task 2: Enrollment Service and Hierarchical DI

src/app/services/enrollment.service.ts (Service-to-Service Injection)

```
import { Injectable } from '@angular/core';
import { CourseService } from '../course.service';
import { Course } from '../models/course.model';

@Injectable({
  providedIn: 'root'
})
export class EnrollmentService {
  private enrolledCourseIds: number[] = [];

  constructor(private courseService: CourseService) {}

  enroll(courseId: number): void {
    if (!this.enrolledCourseIds.includes(courseId)) {
      this.enrolledCourseIds.push(courseId);
    }
  }

  unenroll(courseId: number): void {
    this.enrolledCourseIds = this.enrolledCourseIds.filter(id => id !== courseId);
  }

  isEnrolled(courseId: number): boolean {
    return this.enrolledCourseIds.includes(courseId);
  }

  getEnrolledCourses(): Course[] {
    const allCourses = this.courseService.getCourses();
    return allCourses.filter(c => this.enrolledCourseIds.includes(c.id));
  }
}
```

Component Scoped Isolated Provider Implementation

```
import { Component } from '@angular/core';
import { NotificationService } from '../../services/notification.service';

@Component({
  selector: 'app-notification-widget',
  standalone: true,
  template: '<div class="box">System Notifications Monitor active.</div>',
  providers: [NotificationService] // Explicit scoped component level provider
})
export class NotificationComponent {
  constructor(private notifyService: NotificationService) {}
}
```

DI Concept: Singleton (root) vs Scoped Providers

Declaring `providedIn: 'root'` registers the service inside the root application injector, creating a single application-wide instance. In contrast, declaring a service within a component's `providers: [...]` array allocates a new instance scoped to that component's injector lifecycle, isolating its state changes from the rest of the application.

HANDS-ON 7: Angular Routing - Guards, Lazy Loading & Route Data

Task 1: Route Configuration, Parameters and Nested Routes

Configures target navigation coordinates via parameterized segments, global fallbacks, query mappings, and layout structural routing systems.

src/app/app.routes.ts

```
import { Routes } from '@angular/router';
import { HomeComponent } from '../pages/home/home.component';
import { CoursesLayoutComponent } from '../pages/courses-layout/courses-layout.component';
import { CourseListComponent } from '../pages/course-list/course-list.component';
import { CourseDetailComponent } from '../pages/course-detail/course-detail.component';
import { StudentProfileComponent } from '../pages/student-profile/student-profile.component';
import { NotFoundComponent } from '../pages/not-found/not-found.component';
import { AuthGuard } from '../guards/auth.guard';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  {
    path: 'courses',
    component: CoursesLayoutComponent,
    children: [
      { path: '', component: CourseListComponent },
      { path: ':id', component: CourseDetailComponent }
    ]
  },
  { path: 'profile', component: StudentProfileComponent, canActivate: [AuthGuard] },
  {
    path: 'enroll',
    loadChildren: () => import('../features/enrollment/enrollment.routes').then(m =>
m.ENROLLMENT_ROUTES)
  },
  { path: '**', component: NotFoundComponent }
];
```

src/app/pages/course-detail/course-detail.component.ts

```
import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { CourseService } from '../../services/course.service';
import { Course } from '../../models/course.model';
import { CommonModule } from '@angular/common';

@Component({
  selector: 'app-course-detail',
  standalone: true,
  imports: [CommonModule],
  template: `
    <div *ngIf="course" class="detail-box">
```

```

        <h2>{{ course.name }} Detailed View</h2>
        <p><strong>Code Mapping Index:</strong> {{ course.code }}</p>
        <p><strong>Weight Units:</strong> {{ course.credits }} Credits</p>
        <button (click)="goBack()">Return to Catalog</button>
    </div>
    ,
  })
  export class CourseDetailComponent implements OnInit {
    course: Course | undefined;

    constructor(
      private route: ActivatedRoute,
      private router: Router,
      private courseService: CourseService
    ) {}

    ngOnInit(): void {
      const idParam = this.route.snapshot.paramMap.get('id');
      if (idParam) {
        this.course = this.courseService.getCourseById(Number(idParam));
      }
    }

    goBack(): void {
      this.router.navigate(['/courses']);
    }
  }
}

```

Task 2: Lazy Loading and Route Guards

src/app/guards/auth.guard.ts

```

import { Injectable } from '@angular/core';
import { CanActivate, Router, UrlTree } from '@angular/router';

@Injectable({
  providedIn: 'root'
})
export class AuthGuard implements CanActivate {
  private isLoggedIn: boolean = false; // Mock Authentication Flag

  constructor(private router: Router) {}

  canActivate(): boolean | UrlTree {
    if (this.isLoggedIn) {
      return true;
    }
    // Force rerouting to secure home coordinates on validation failure
    return this.router.parseUrl('/');
  }
}

```

src/app/guards/unsaved-changes.guard.ts

```
import { Injectable } from '@angular/core';
import { CanDeactivate } from '@angular/router';
import { ReactiveEnrollmentFormComponent } from '../pages/reactive-enrollment-form/reactive-enrollment-form.component';

@Injectable({
  providedIn: 'root'
})
export class UnsavedChangesGuard implements CanDeactivate<ReactiveEnrollmentFormComponent> {
  canDeactivate(component: ReactiveEnrollmentFormComponent): boolean {
    if (component.enrollForm && component.enrollForm.dirty) {
      return window.confirm('You have unsaved form structural entries. Do you intend to discard changes?');
    }
    return true;
  }
}
```


HANDS-ON 8: HTTP Client - API Integration, Observables & Interceptors

Task 1: Replace Service Data with HttpClient Calls

Transition hardcoded vectors into runtime network processing pipelines using `HttpClient` async operations linked to a mock REST server.

src/app/services/course.service.ts (Refactored to live API)

```
import { Injectable } from '@angular/core';
import { HttpClient } from '@angular/common/http';
import { Observable, throwError } from 'rxjs';
import { map, catchError, tap, retry } from 'rxjs/operators';
import { Course } from '../models/course.model';

@Injectable({
  providedIn: 'root'
})
export class CourseService {
  private apiUrl = 'http://localhost:3000/courses';

  constructor(private http: HttpClient) {}

  getCourses(): Observable<Course[]> {
    return this.http.get<Course[]>(this.apiUrl).pipe(
      retry(2),
      map(courses => courses.filter(c => c.credits > 0)),
      tap(courses => console.log(`Pipeline streaming completed. Count: ${courses.length}`)),
      catchError(this.handleError)
    );
  }

  getCourseById(id: number): Observable<Course> {
    return this.http.get<Course>(`${this.apiUrl}/${id}`).pipe(
      catchError(this.handleError)
    );
  }

  createCourse(course: Omit<Course, 'id'>): Observable<Course> {
    return this.http.post<Course>(this.apiUrl, course).pipe(
      catchError(this.handleError)
    );
  }

  updateCourse(id: number, course: Course): Observable<Course> {
    return this.http.put<Course>(`${this.apiUrl}/${id}`, course).pipe(
      catchError(this.handleError)
    );
  }

  deleteCourse(id: number): Observable<void> {
    return this.http.delete<void>(`${this.apiUrl}/${id}`).pipe(
      catchError(this.handleError)
    );
  }
}
```

```

    );
  }

  private handleError(error: any) {
    console.error('Network infrastructure crash:', error);
    return throwError(() => new Error('Failed to process course repository action. Try again later.'));
  }
}

```

Task 2: RxJS Operators and Error Handling

RxJS Pipeline Orchestration Explanations

tap: Used to execute debugging logs or analytics side effects without mutating the values in the stream.

switchMap: Essential for handling changing parameters (like dynamic search inputs). When a new search term or ID arrives, `switchMap` automatically unsubscribes from the previous pending inner request and switches to the new one, avoiding race conditions or out-of-order responses.

Task 3: HTTP Interceptors

src/app/interceptors/auth.interceptor.ts

```

import { HttpInterceptorFn } from '@angular/common/http';

export const authInterceptor: HttpInterceptorFn = (req, next) => {
  const authToken = 'Bearer mock-token-12345';
  const secureReq = req.clone({
    headers: { Authorization: authToken }
  });
  return next(secureReq);
};

```

src/app/interceptors/loading.interceptor.ts

```

import { HttpInterceptorFn } from '@angular/common/http';
import { inject } from '@angular/core';
import { LoadingService } from '../../services/loading.service';
import { finalize } from 'rxjs/operators';

export const loadingInterceptor: HttpInterceptorFn = (req, next) => {
  const loadingService = inject(LoadingService);
  loadingService.showSpinner();

  return next(req).pipe(
    finalize(() => {
      loadingService.hideSpinner();
    })
  );
};

```

```
);  
};
```

HANDS-ON 9: State Management - NgRx Store, Actions, Reducers, Effects & Selectors

Task 1 & Task 2: Architecture Framework & Core Segments

Migrates localized state storage from individual services into a centralized, immutable global data repository governed by the Redux design pattern.

src/app/store/course/course.actions.ts

```
import { createAction, props } from '@ngrx/store';
import { Course } from '../../../models/course.model';

export const loadCourses = createAction('[Course Catalog] Load Courses Trigger');
export const loadCoursesSuccess = createAction('[Course Catalog] Load Success',
  props<{ courses: Course[] }>());
export const loadCoursesFailure = createAction('[Course Catalog] Load Failure', props<{ error:
  string }>());
```

src/app/store/course/course.reducer.ts

```
import { createReducer, on } from '@ngrx/store';
import * as CourseActions from './course.actions';
import { Course } from '../../../models/course.model';

export interface CourseState {
  courses: Course[];
  loading: boolean;
  error: string | null;
}

export const initialState: CourseState = {
  courses: [],
  loading: false,
  error: null
};

export const courseReducer = createReducer(
  initialState,
  on(CourseActions.loadCourses, state => ({ ...state, loading: true, error: null })),
  on(CourseActions.loadCoursesSuccess, (state, { courses }) => ({ ...state, loading: false,
    courses })),
  on(CourseActions.loadCoursesFailure, (state, { error }) => ({ ...state, loading: false,
    error })))
);
```

src/app/store/course/course.selectors.ts

```
import { createFeatureSelector, createSelector } from '@ngrx/store';
import { CourseState } from './course.reducer';
```

```
export const selectCourseState = createFeatureSelector<CourseState>('course');

export const selectAllCourses = createSelector(selectCourseState, state => state.courses);
export const selectCoursesLoading = createSelector(selectCourseState, state => state.loading);
```

src/app/store/course/course.effects.ts

```
import { Injectable } from '@angular/core';
import { Actions, createEffect, ofType } from '@ngrx/effects';
import { CourseService } from '../../../services/course.service';
import * as CourseActions from './course.actions';
import { mergeMap, map, catchError } from 'rxjs/operators';
import { of } from 'rxjs';

@Injectable()
export class CourseEffects {
  loadCourses$ = createEffect(() =>
    this.actions$.pipe(
      ofType(CourseActions.loadCourses),
      mergeMap(() =>
        this.courseService.getCourses().pipe(
          map(courses => CourseActions.loadCoursesSuccess({ courses })),
          catchError(err => of(CourseActions.loadCoursesFailure({ error: err.message })))
        )
      )
    )
  );

  constructor(private actions$: Actions, private courseService: CourseService) {}
}
```

NgRx Best Practice: Cross-Slice Feature Selectors

When building enterprise workflows, data segments must remain normalized. Joining distinct feature spaces (e.g., matching IDs in the enrollment slice against full course definitions in the course slice) should be handled inside cross-slice selectors using `createSelector` with multiple inputs, keeping reducers pure and computational logic highly reusable.

HANDS-ON 10: Unit Testing Angular Applications - Jasmine, Karma & TestBed

Task 1: Testing a Component - CourseCardComponent

Verifies isolated view presentation logic, property injection states, and output event triggers using Angular test beds.

src/app/components/course-card/course-card.component.spec.ts

```
import { TestBed, ComponentFixture } from '@angular/core/testing';
import { CourseCardComponent } from './course-card.component';
import { By } from '@angular/platform-browser';
import { CreditLabelPipe } from '../../pipes/credit-label.pipe';
import { SimpleChange } from '@angular/core';

describe('CourseCardComponent Technical Specification', () => {
  let component: CourseCardComponent;
  let fixture: ComponentFixture<CourseCardComponent>;

  beforeEach(async () => {
    await TestBed.configureTestingModule({
      imports: [CourseCardComponent, CreditLabelPipe]
    }).compileComponents();

    fixture = TestBed.createComponent(CourseCardComponent);
    component = fixture.componentInstance;
  });

  it('should instantiate the component profile successfully', () => {
    expect(component).toBeTruthy();
  });

  it('should render course name input in template title text structure', () => {
    component.course = { id: 1, name: 'Cloud Native Microservices', code: 'CL-A09', credits: 3,
    gradeStatus: 'pending' };
    fixture.detectChanges();

    const titleEl = fixture.debugElement.query(By.css('h3')).nativeElement;
    expect(titleEl.textContent).toContain('Cloud Native Microservices');
  });

  it('should emit target numeric identifier payload on enrollment click', () => {
    component.course = { id: 505, name: 'Test Course', code: 'TC-01', credits: 4, gradeStatus:
    'passed' };
    fixture.detectChanges();

    spyOn(component.enrollRequested, 'emit');
    const button = fixture.debugElement.query(By.css('.btn-enroll')).nativeElement;
    button.click();

    expect(component.enrollRequested.emit).toHaveBeenCalled();
  });
});
```

```
});  
});
```

Task 2: Testing a Service and an NgRx-Connected Component

src/app/services/course.service.spec.ts

```
import { TestBed } from '@angular/core/testing';  
import { HttpClientTestingModule, HttpTestingController } from '@angular/common/http/testing';  
import { CourseService } from './course.service';  
import { Course } from '../models/course.model';  
  
describe('CourseDataService Integration Infrastructure', () => {  
  let service: CourseService;  
  let httpMock: HttpTestingController;  
  
  beforeEach(() => {  
    TestBed.configureTestingModule({  
      imports: [HttpClientTestingModule],  
      providers: [CourseService]  
    });  
    service = TestBed.inject(CourseService);  
    httpMock = TestBed.inject(HttpTestingController);  
  });  
  
  afterEach(() => {  
    httpMock.verify(); // Validate that no outstanding requests remain open  
  });  
  
  it('should request and return filtered course data vectors from the server target API', () => {  
    {  
      const mockPayload: Course[] = [  
        { id: 1, name: 'Angular Unit Testing Architecture', code: 'TS-01', credits: 3,  
gradeStatus: 'passed' }  
      ];  
  
      service.getCourses().subscribe(data => {  
        expect(data.length).toBe(1);  
        expect(data[0].name).toEqual('Angular Unit Testing Architecture');  
      });  
  
      const req = httpMock.expectOne('http://localhost:3000/courses');  
      expect(req.request.method).toBe('GET');  
      req.flush(mockPayload);  
    }  
  });  
});
```